

```
In [1]: # Importing essential data manipulation and visualization libraries for the analysis.
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

In [2]: # Loading the employee performance dataset and taking a quick look at the first few rows.
data=pd.read_csv("C:\\Users\\Dell\\OneDrive\\Desktop\\Project 1\\Extended_Employee_Performance
data
```

Out[2]:

	Employee_ID	Department	Gender	Age	Job_Title	Hire_Date	Years_At_Company	Educa
0	1	IT	Male	55	Specialist	2022-01-19 08:03:05.556036	2	H
1	2	Finance	Male	29	Developer	2024-04-18 08:03:05.556036	0	H
2	3	Finance	Male	55	Specialist	2015-10-26 08:03:05.556036	8	H
3	4	Customer Support	Female	48	Analyst	2016-10-22 08:03:05.556036	7	
4	5	Engineering	Female	36	Analyst	2021-07-23 08:03:05.556036	3	
...	...	...	...	...	...	...	...	
99995	99996	Finance	Male	27	Technician	2022-12-07 08:03:05.556036	1	
99996	99997	IT	Female	36	Consultant	2018-07-24 08:03:05.556036	6	
99997	99998	Operations	Male	53	Analyst	2015-11-24 08:03:05.556036	8	H
99998	99999	HR	Female	22	Consultant	2015-08-03 08:03:05.556036	9	H
99999	100000	Finance	Female	43	Analyst	2024-03-04 08:03:05.556036	0	

100000 rows × 20 columns

```
In [3]: # Inspecting the tail end of the dataframe to verify record structures and indexing.
data.tail()
```

Out[3]:

	Employee_ID	Department	Gender	Age	Job_Title	Hire_Date	Years_At_Company	Educa
	99995	99996	Finance	Male	27	Technician	2022-12-07 08:03:05.556036	1
	99996	99997	IT	Female	36	Consultant	2018-07-24 08:03:05.556036	6
	99997	99998	Operations	Male	53	Analyst	2015-11-24 08:03:05.556036	8
	99998	99999	HR	Female	22	Consultant	2015-08-03 08:03:05.556036	9
	99999	100000	Finance	Female	43	Analyst	2024-03-04 08:03:05.556036	0

In [4]:

```
# Checking for any duplicate employee records to ensure data integrity.  
data["Employee_ID"].duplicated().sum()
```

Out[4]: np.int64(0)

In [5]:

```
# Checking total rows, columns, and data types to identify any missing or mismatched values.  
print("Data Column & Rows",data.shape)  
print(data.info())
```

Data Column & Rows (100000, 20)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 20 columns):
Column Non-Null Count Dtype
--- -
0 Employee_ID 100000 non-null int64
1 Department 100000 non-null object
2 Gender 100000 non-null object
3 Age 100000 non-null int64
4 Job_Title 100000 non-null object
5 Hire_Date 100000 non-null object
6 Years_At_Company 100000 non-null int64
7 Education_Level 100000 non-null object
8 Performance_Score 100000 non-null int64
9 Monthly_Salary 100000 non-null float64
10 Work_Hours_Per_Week 100000 non-null int64
11 Projects_Handled 100000 non-null int64
12 Overtime_Hours 100000 non-null int64
13 Sick_Days 100000 non-null int64
14 Remote_Work_Frequency 100000 non-null int64
15 Team_Size 100000 non-null int64
16 Training_Hours 100000 non-null int64
17 Promotions 100000 non-null int64
18 Employee_Satisfaction_Score 100000 non-null float64
19 Resigned 100000 non-null bool
dtypes: bool(1), float64(2), int64(12), object(5)
memory usage: 14.6+ MB
None

In [6]:

```
# Generating statistical summary metrics for numerical features to spot outliers or anomalies  
data.describe()
```

Out[6]:

	Employee_ID	Age	Years_At_Company	Performance_Score	Monthly_Salary	Work_Hc
count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	
mean	50000.500000	41.029410	4.476070	2.995430	6403.211000	
std	28867.657797	11.244121	2.869336	1.414726	1372.508717	
min	1.000000	22.000000	0.000000	1.000000	3850.000000	
25%	25000.750000	31.000000	2.000000	2.000000	5250.000000	
50%	50000.500000	41.000000	4.000000	3.000000	6500.000000	
75%	75000.250000	51.000000	7.000000	4.000000	7500.000000	
max	100000.000000	60.000000	10.000000	5.000000	9000.000000	

In [7]:

```
# Cleaning up column names by removing underscores for better readability and querying.
data.columns=data.columns.str.replace("_", " ")
data.columns
```

Out[7]:

```
Index(['Employee ID', 'Department', 'Gender', 'Age', 'Job Title', 'Hire Date',
      'Years At Company', 'Education Level', 'Performance Score',
      'Monthly Salary', 'Work Hours Per Week', 'Projects Handled',
      'Overtime Hours', 'Sick Days', 'Remote Work Frequency', 'Team Size',
      'Training Hours', 'Promotions', 'Employee Satisfaction Score',
      'Resigned'],
      dtype='object')
```

In [8]:

```
# Re-verifying column data types after preprocessing steps.
data.dtypes
```

Out[8]:

```
Employee ID          int64
Department          object
Gender              object
Age                 int64
Job Title           object
Hire Date           object
Years At Company    int64
Education Level     object
Performance Score    int64
Monthly Salary      float64
Work Hours Per Week int64
Projects Handled    int64
Overtime Hours      int64
Sick Days           int64
Remote Work Frequency int64
Team Size           int64
Training Hours      int64
Promotions          int64
Employee Satisfaction Score float64
Resigned            bool
dtype: object
```

In [9]:

```
# Counting total employee distribution across individual company departments.
data["Department"].value_counts()
```

```
Out[9]: Department
Marketing      11216
Finance        11200
Operations     11181
IT             11131
Sales          11122
Legal          11118
Customer Support 11116
HR             10960
Engineering    10956
Name: count, dtype: int64
```

```
In [10]: # Calculating high-level workforce KPIs like overall headcount, attrition rate, and averages.
total_employee=data["Employee ID"].count()
attrition_rate=data["Resigned"].sum()/total_employee
avg_monthly_salary=data["Monthly Salary"].mean()
avg_satisfaction_score=data["Employee Satisfaction Score"].mean()
avg_performance_score=data["Performance Score"].mean()
avg_overtime_hours=data["Overtime Hours"].mean()
avg_tenure_yrs=data["Years At Company"].mean()
total_department=data["Department"].nunique()
```

```
In [11]: # Displaying high-level workforce KPIs like overall headcount, attrition rate, and averages.
print("="*40)
print("""-----Workforce Overview &-----
-----Key Performance Indicators-----""")
print("="*40)
print()
print(f"Total Employees: {total_employee}")
print(f"Number of Departments: ",total_department)
print(f"Average Tenure Years: {avg_tenure_yrs:,.2f}Y")
print(f"Attrition Rate: {attrition_rate*100:,.2f}%")
print(f"Average Monthly Salary: ${avg_monthly_salary:,.2f}")
print(f"Average Satisfaction Score: {round(avg_satisfaction_score):,.2f}")
print(f"Average Performance Score: {round(avg_performance_score):,.2f}")
print(f"Average Overtime Hours: {avg_overtime_hours:,.2f} Hours")
```

```
=====
-----Workforce Overview &-----
-----Key Performance Indicators-----
=====
```

```
Total Employees: 100000
Number of Departments: 9
Average Tenure Years: 4.48Y
Attrition Rate: 10.01%
Average Monthly Salary: $6,403.21
Average Satisfaction Score: 3.00
Average Performance Score: 3.00
Average Overtime Hours: 14.51 Hours
```

```
In [12]: data.columns
```

```
Out[12]: Index(['Employee ID', 'Department', 'Gender', 'Age', 'Job Title', 'Hire Date',
               'Years At Company', 'Education Level', 'Performance Score',
               'Monthly Salary', 'Work Hours Per Week', 'Projects Handled',
               'Overtime Hours', 'Sick Days', 'Remote Work Frequency', 'Team Size',
               'Training Hours', 'Promotions', 'Employee Satisfaction Score',
               'Resigned'],
              dtype='object')
```

```
In [13]: # Visualizing the employee distribution across different departments to check headcount balance
plt.figure(figsize=(14,6))
category_dep=data.groupby("Department")["Employee ID"].count().sort_values(ascending=False).r
```

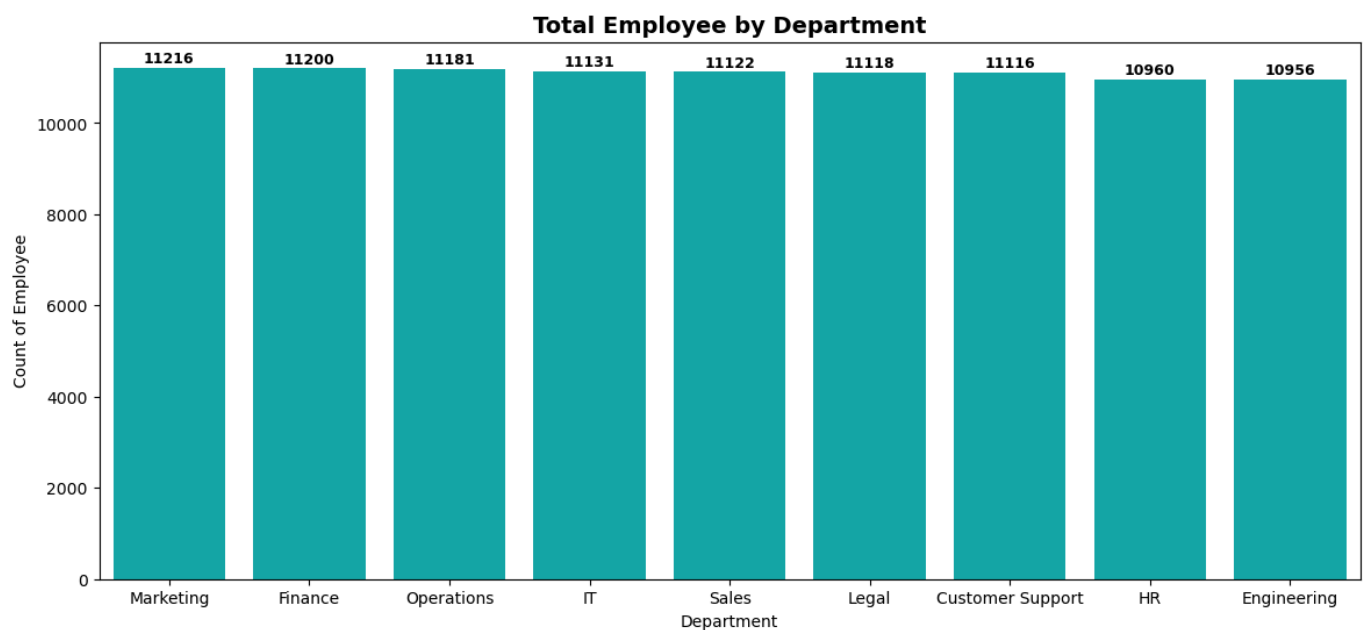
```

ax=sns.barplot(
    x="Department",
    y="Employee ID",
    data=category_dep,
    legend=False,
    color="c"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=1,
        fontweight="bold",
        fontsize=9
    )

plt.title(
    "Total Employee by Department",
    fontsize=14,
    fontweight="bold"
)
plt.ylabel(
    "Count of Employee"
)
plt.xlabel(
    "Department"
)
plt.savefig("01total_employee_by_department.png", dpi=300, bbox_inches="tight")
plt.show()

```



```

In [14]: # Analyzing department-wise attrition rates to identify where employee turnover is highest.
plt.figure(figsize=(14,6))
attrition_rate=(data.groupby("Department")["Resigned"].sum()/
                data.groupby("Department")["Resigned"].count()*100).sort_values(ascending=False)

ax=sns.barplot(
    x="Department",
    y="Attrition_Rate",
    data=attrition_rate,
    legend=False,
    color="cyan"
)

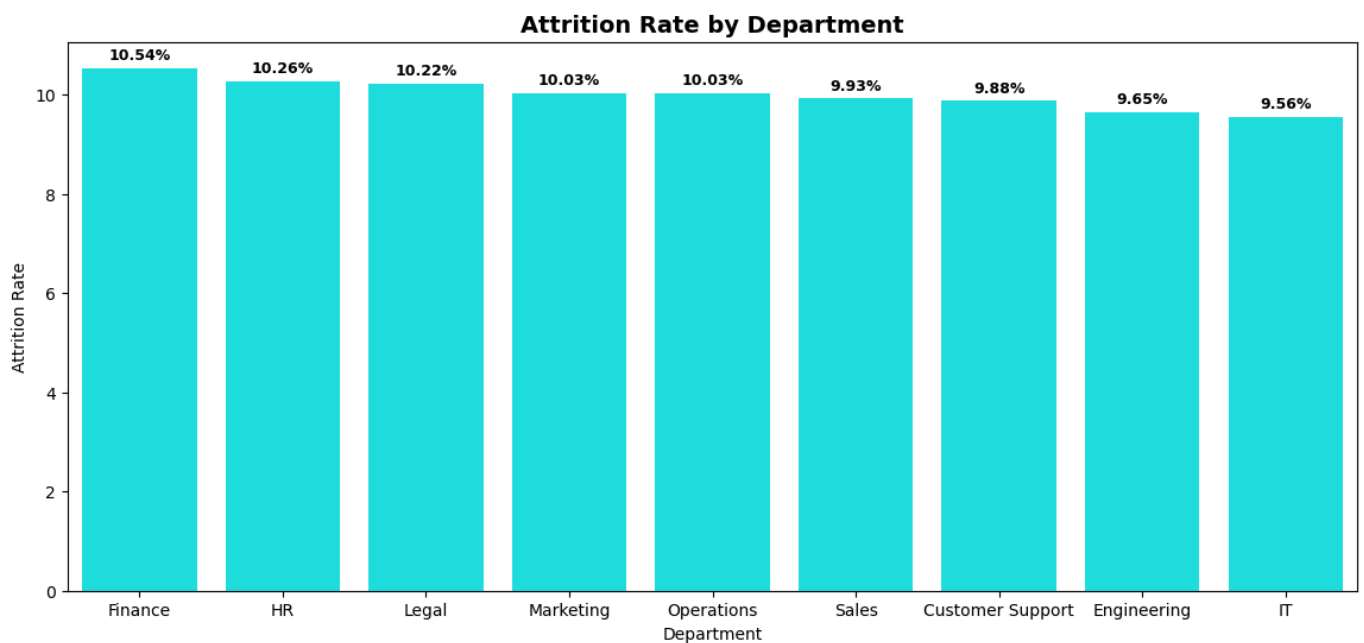
```

```

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.2f%%",
        padding=3,
        fontweight="bold",
        fontsize=9
    )

plt.title(
    "Attrition Rate by Department",
    fontweight="bold",
    fontsize=14
)
plt.xlabel(
    "Department"
)
plt.ylabel(
    "Attrition Rate"
)
plt.savefig("02_attrition_rate_by_department.png", dpi=300, bbox_inches="tight")
plt.show()

```



```

In [15]: # Checking average monthly salaries department-wise to ensure internal pay equity.
plt.figure(figsize=(14,6))
avg_dep_monthly_salary=data.groupby("Department")["Monthly Salary"].mean().sort_values(ascending=True)

ax=sns.barplot(
    x="Department",
    y="Salary",
    data=avg_dep_monthly_salary,
    legend=False,
    color="orchid"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="$%.2f",
        padding=0,
        fontweight="bold",
        fontsize=9
    )

plt.title(

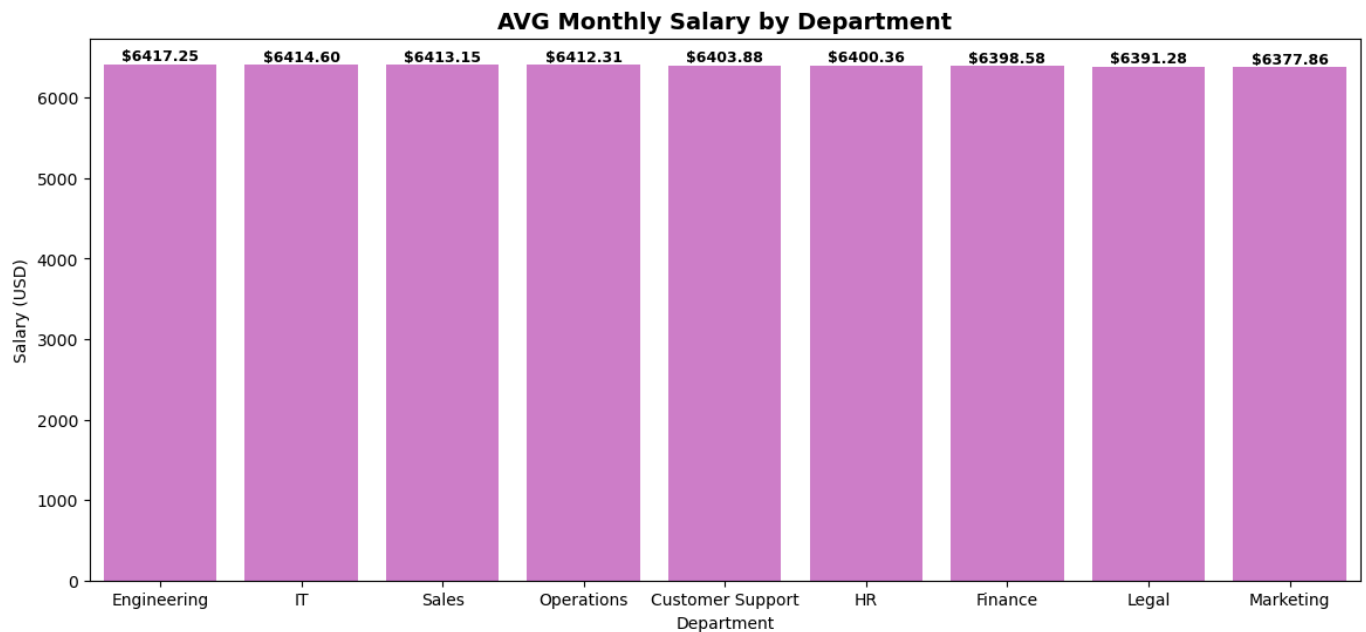
```

```

"AVG Monthly Salary by Department",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Department"
)
plt.ylabel(
    "Salary (USD)"
)

plt.savefig("03_avg_monthly_salary_by_department.png", dpi=300, bbox_inches="tight")
plt.show()

```



```

In [16]: # Comparing average monthly compensation levels grouped by employee education qualifications.
plt.figure(figsize=(10,5))
avg_salary_by_education=data.groupby("Education Level")["Monthly Salary"].mean().sort_values(

ax=sns.barplot(
    x="Education Level",
    y="Salary",
    data=avg_salary_by_education,
    legend=False,
    color="pink"
)

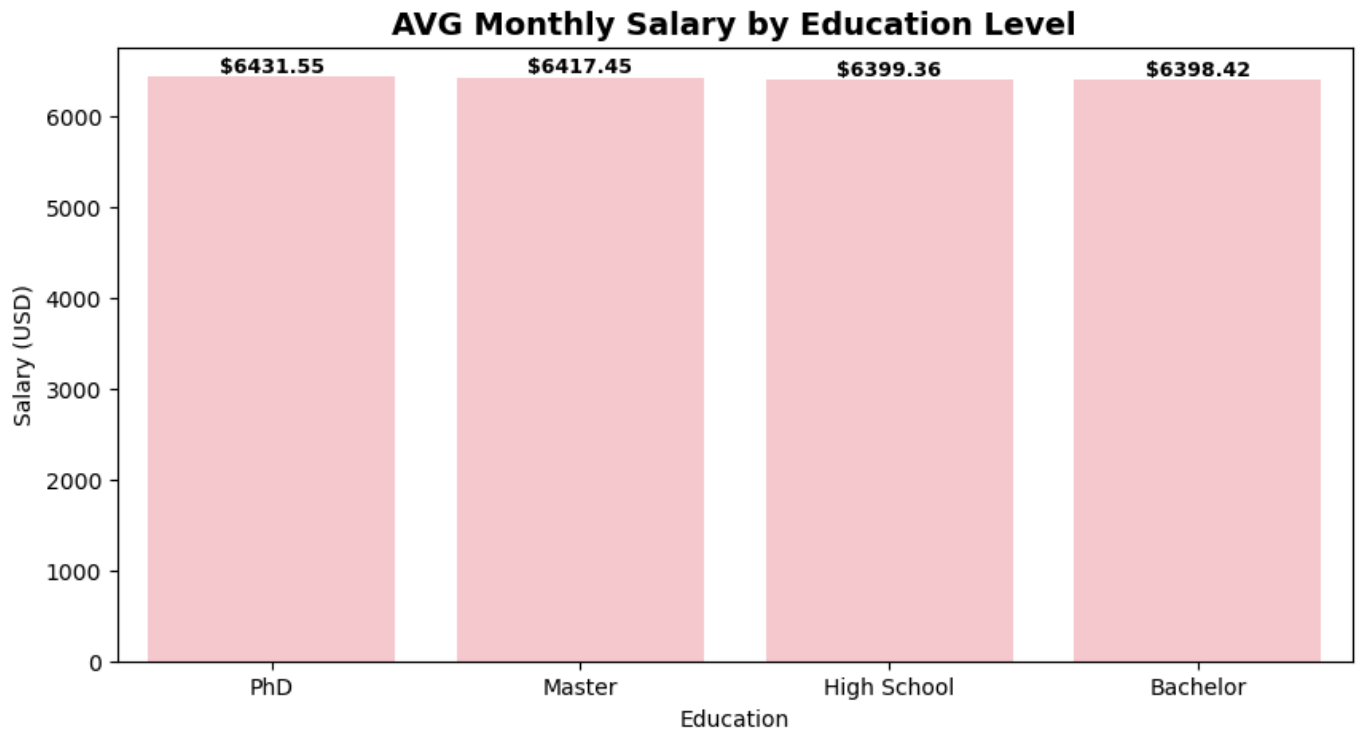
for i in ax.containers:
    ax.bar_label(
        i,
        fmt="$%.2f",
        padding=0,
        fontweight="bold",
        fontsize=9
    )

plt.title(
    "AVG Monthly Salary by Education Level",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Education"
)
plt.ylabel(
    "Salary (USD)"
)

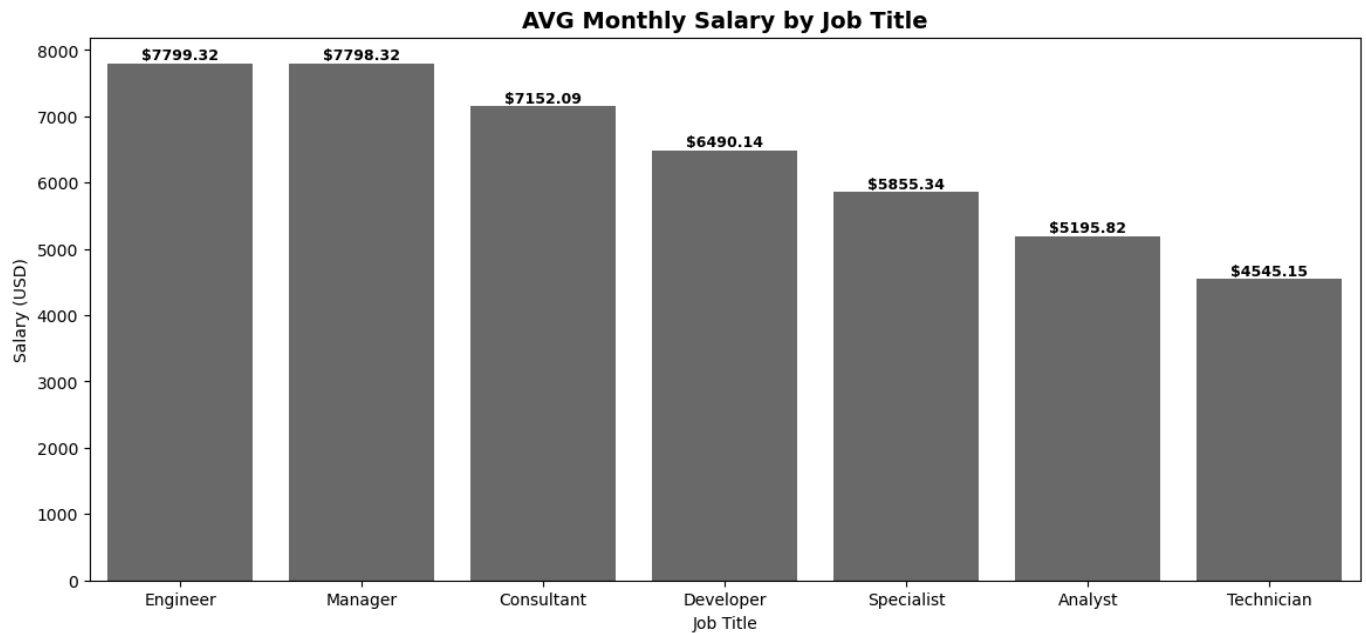
```

```
)
```

```
plt.savefig("04_avg_monthly_salary_by_education.png", dpi=300, bbox_inches="tight")  
plt.show()
```



```
In [17]: # Evaluating salary distribution across various job roles to map compensation structures.  
plt.figure(figsize=(14,6))  
avg_salary_by_job=data.groupby("Job Title")["Monthly Salary"].mean().sort_values(ascending=False)  
  
ax=sns.barplot(  
    x="Job Title",  
    y="Salary",  
    data=avg_salary_by_job,  
    legend=False,  
    color="dimgray"  
)  
  
for i in ax.containers:  
    ax.bar_label(  
        i,  
        fmt="$.2f",  
        padding=0,  
        fontweight="bold",  
        fontsize=9  
    )  
  
plt.title(  
    "AVG Monthly Salary by Job Title",  
    fontsize=14,  
    fontweight="bold"  
)  
plt.xlabel(  
    "Job Title"  
)  
plt.ylabel(  
    "Salary (USD)"  
)  
  
plt.savefig("05_avg_monthly_salary_by_job.png", dpi=300, bbox_inches="tight")  
plt.show()
```

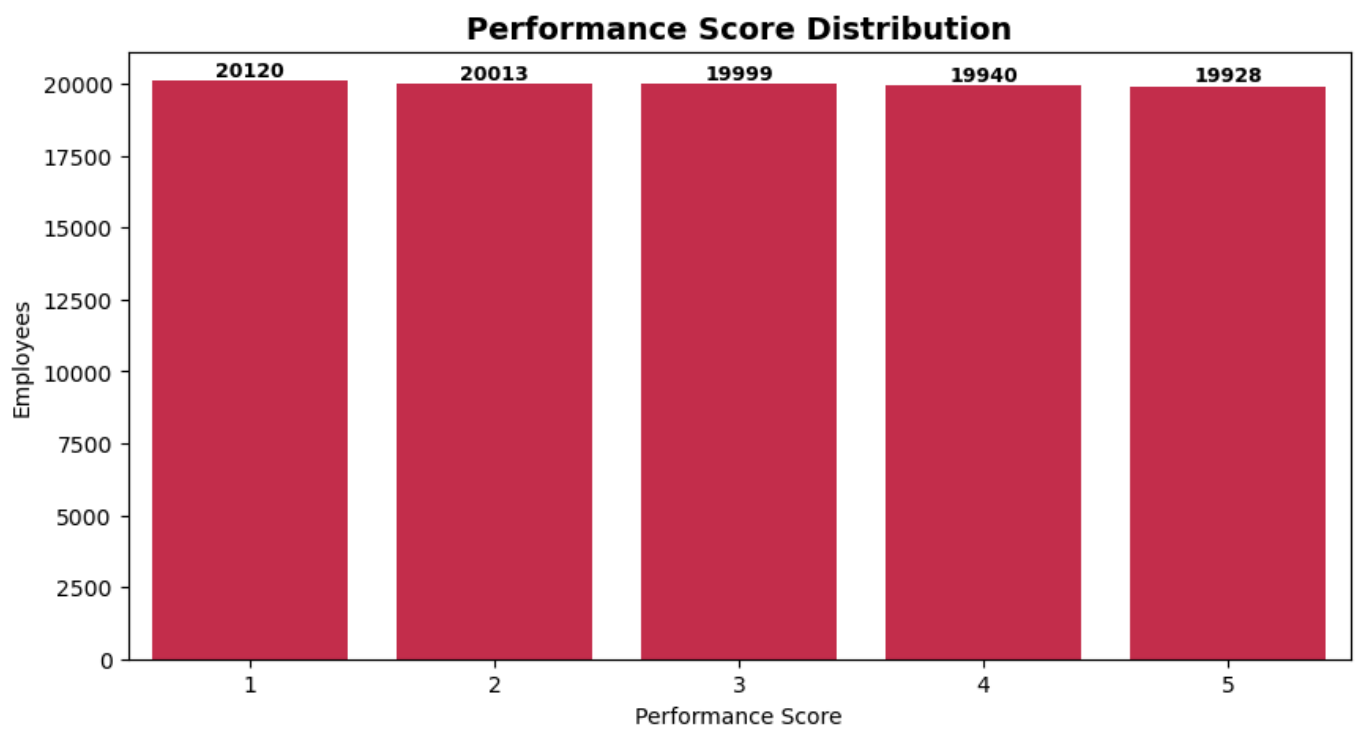
```
In [18]: # Checking the overall distribution spread of employee performance scores.
plt.figure(figsize=(10,5))
performance_score=data.groupby("Performance Score")["Employee ID"].count().sort_values(ascending=True)

ax=sns.barplot(
    x="Performance Score",
    y="Employees",
    data=performance_score,
    legend=False,
    color="crimson"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=0,
        fontweight="bold",
        fontsize=9
    )

plt.title(
    "Performance Score Distribution",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Performance Score"
)
plt.ylabel(
    "Employees"
)

plt.savefig("06_performance_score_distribution.png", dpi=300, bbox_inches="tight")
plt.show()
```



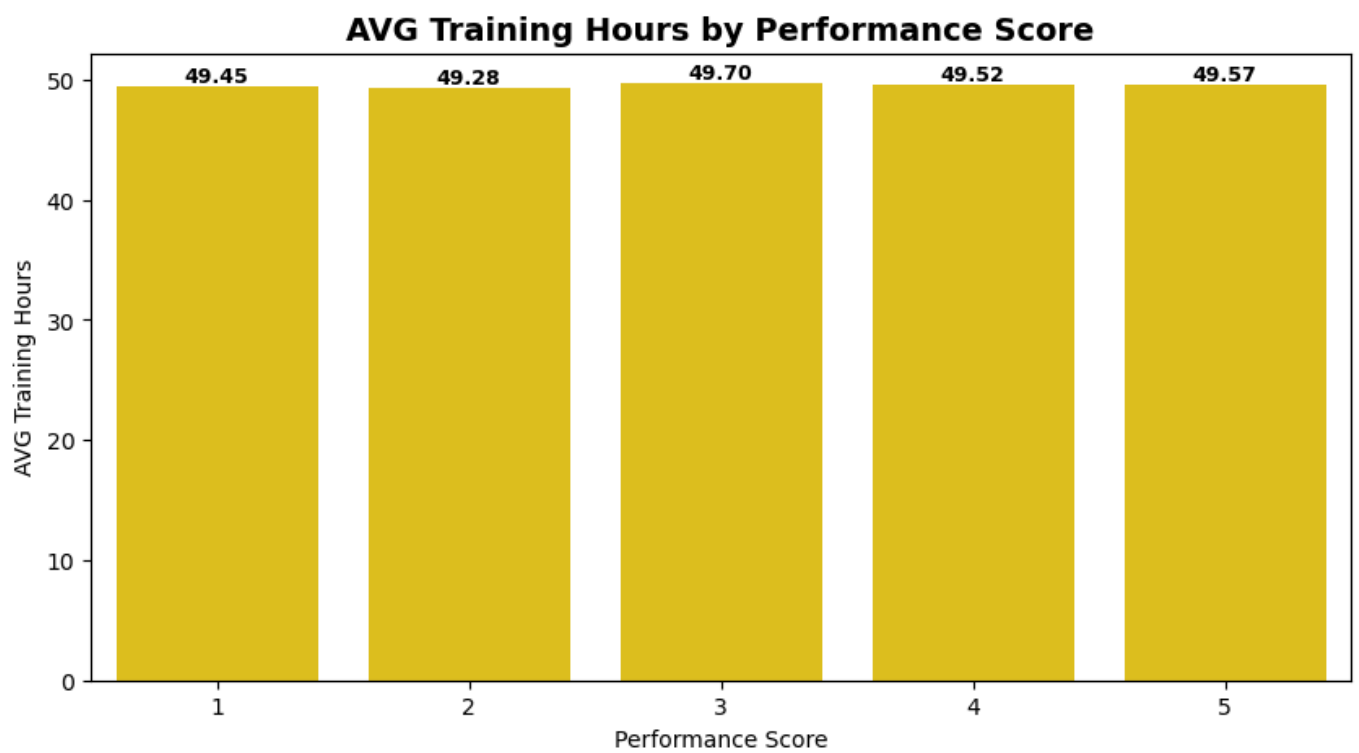
```
In [19]: # Checking if average training hours have any visible impact on employee performance scores.
plt.figure(figsize=(10,5))
avg_training_performance_score=data.groupby("Performance Score")["Training Hours"].mean().sort()

ax=sns.barplot(
    x="Performance Score",
    y="Training",
    data=avg_training_performance_score,
    legend=False,
    color="gold"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.2f",
        padding=0,
        fontweight="bold",
        fontsize=9
    )

plt.title(
    "AVG Training Hours by Performance Score",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Performance Score"
)
plt.ylabel(
    "AVG Training Hours"
)

plt.savefig("07_avg_training_hours_by_performance_score.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
In [20]: # Plotting employee satisfaction score distribution to gauge overall workforce sentiment.
plt.figure(figsize=(6,4))

# Creating satisfaction score bins/ranges to categorize employee sentiment levels.
bins=[0,1,2,3,4,5]
labels=[1,2,3,4,5]
data["Satisfaction Range"]=pd.cut(
    data["Employee Satisfaction Score"],
    bins=bins,
    labels=labels,
    include_lowest=True,
)

employee_satisfaction_score=data.groupby("Satisfaction Range", observed=False)["Employee ID"]

ax=sns.barplot(
    x="Satisfaction Range",
    y="Employees",
    data=employee_satisfaction_score,
    legend=False,
    color="gold"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=0,
        fontweight="bold",
        fontsize=9
    )

plt.title(
    "Employee Satisfaction Score",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Satisfaction Score"
)
plt.ylabel(
    "Employee"
```

```
)

plt.savefig("08_employee_satisfaction_score.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
In [21]: # Mapping and comparing satisfaction scores between active employees and those who resigned.
def resign(x):
    if x:
        return "Resigned"
    else:
        return "Stayed"
data["Resignation"]=data["Resigned"].apply(resign)

avg=data.groupby("Resignation")["Employee Satisfaction Score"].mean().sort_values(ascending=False)
avg
```

```
Out[21]:
```

	Resignation	Average Score
0	Stayed	3.000174
1	Resigned	2.989325

```
In [22]: # Comparing satisfaction scores between employees who stayed versus those who resigned.
plt.figure(figsize=(6,4))

ax=sns.barplot(
    x="Resignation",
    y="Average Score",
    data=avg,
    legend=False,
    color="lightpink"
);

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.2f",
        padding=0,
        fontweight="bold",
        fontsize=9
    )
```

```

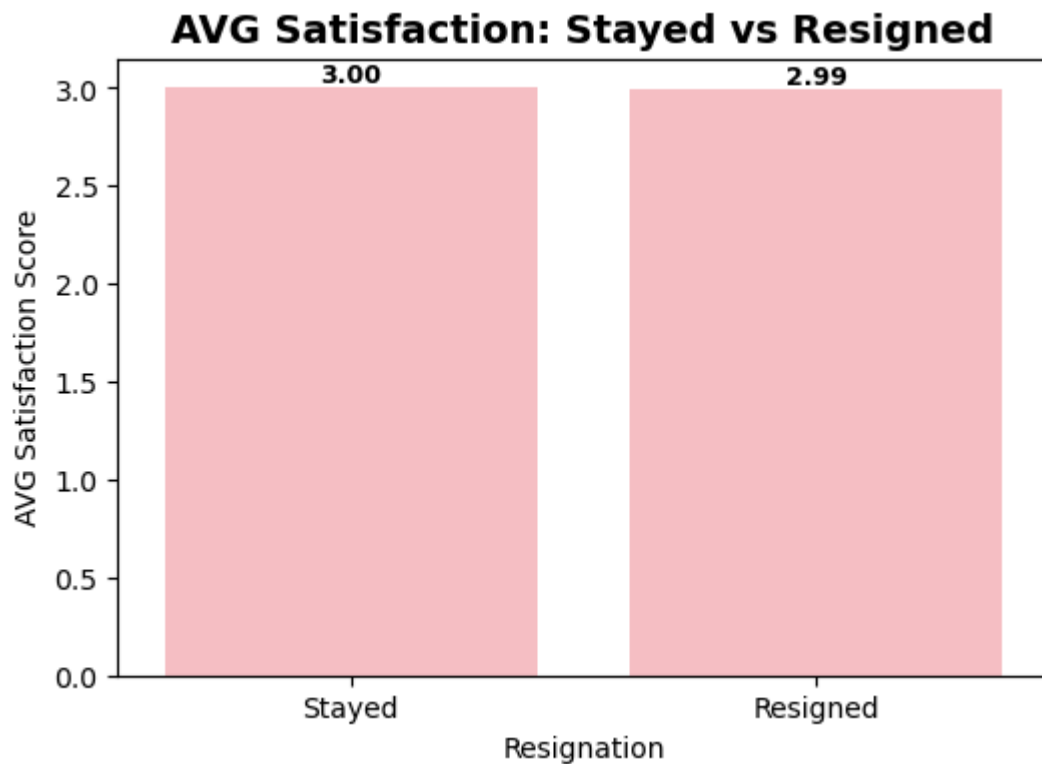
)

plt.title(
    "AVG Satisfaction: Stayed vs Resigned",
    fontweight="bold",
    fontsize=14
)

plt.ylabel(
    "AVG Satisfaction Score"
)

plt.savefig("09_avg_satisfaction_resignation.png", dpi=300, bbox_inches="tight")
plt.show()

```



```

In [23]: attrition_rate_freq=(data.groupby("Remote Work Frequency")["Resigned"].sum()/
        data.groupby("Remote Work Frequency")["Resigned"].count()*100).sort_values(ascending=True)
attrition_rate_freq

```

```

Out[23]:
   Remote Work Frequency  Attrition_Rate_freq
0                      75          10.281069
1                      100          10.165096
2                       25          10.111193
3                       50          10.002030
4                       0           9.482281

```

```

In [24]: # Tracking how varying remote work frequency percentages affect overall attrition rates.
plt.figure(figsize=(8,5))

```

```

sns.lineplot(
    x="Remote Work Frequency",
    y="Attrition_Rate_freq",
    data=attrition_rate_freq,
    legend=False,
    marker="o",
    linewidth=3,
    markersize=9,
)

```

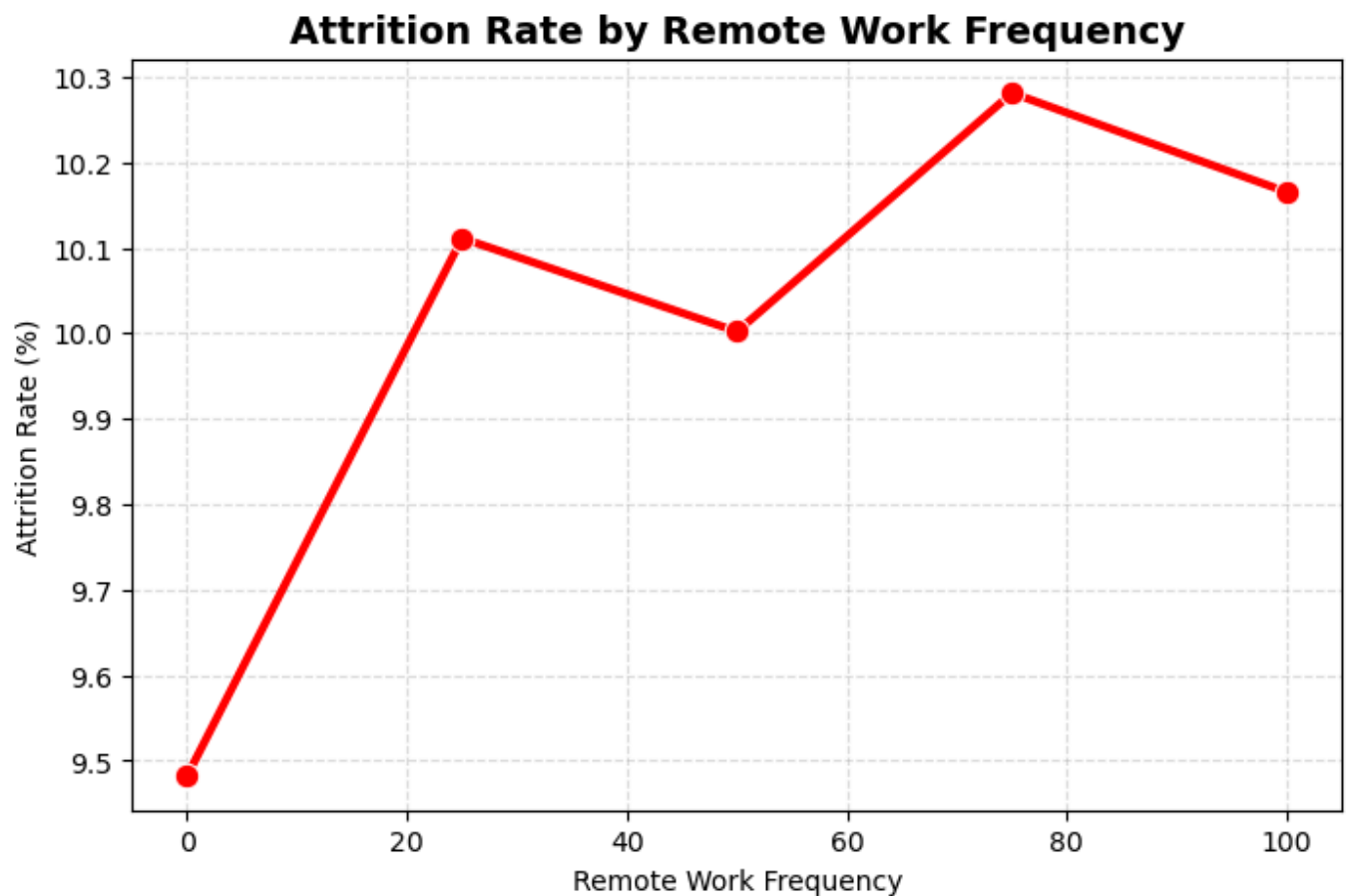
```

        color="red"
    )
    plt.grid(True, linestyle="--", alpha=0.4)

    plt.title(
        "Attrition Rate by Remote Work Frequency",
        fontsize=14,
        fontweight="bold"
    )
    plt.ylabel(
        "Attrition Rate (%)"
    )
    plt.xlabel(
        "Remote Work Frequency"
    )

    plt.savefig("10_attrition_rate_by_remote_work_frequency.png", dpi=300, bbox_inches="tight")
    plt.show()

```



```

In [25]: avg_salary_by_gender=data.groupby("Gender")["Monthly Salary"].mean().sort_values(ascending=False)
         avg_salary_by_gender

```

```

Out[25]:
   Gender  avg_salary
0   Other  6414.768145
1  Female  6404.986354
2   Male   6400.481980

```

```

In [26]: # Breaking down average monthly salaries by gender to monitor compensation parity.
         plt.figure(figsize=(6,4))

         ax=sns.barplot(
             x="Gender",
             y="avg_salary",
             data=avg_salary_by_gender,

```

```

        legend=False,
        color="orange"
    )

    for i in ax.containers:
        ax.bar_label(
            i,
            fmt="$.2f",
            padding=0,
            fontsize=9,
            fontweight="bold"
        )

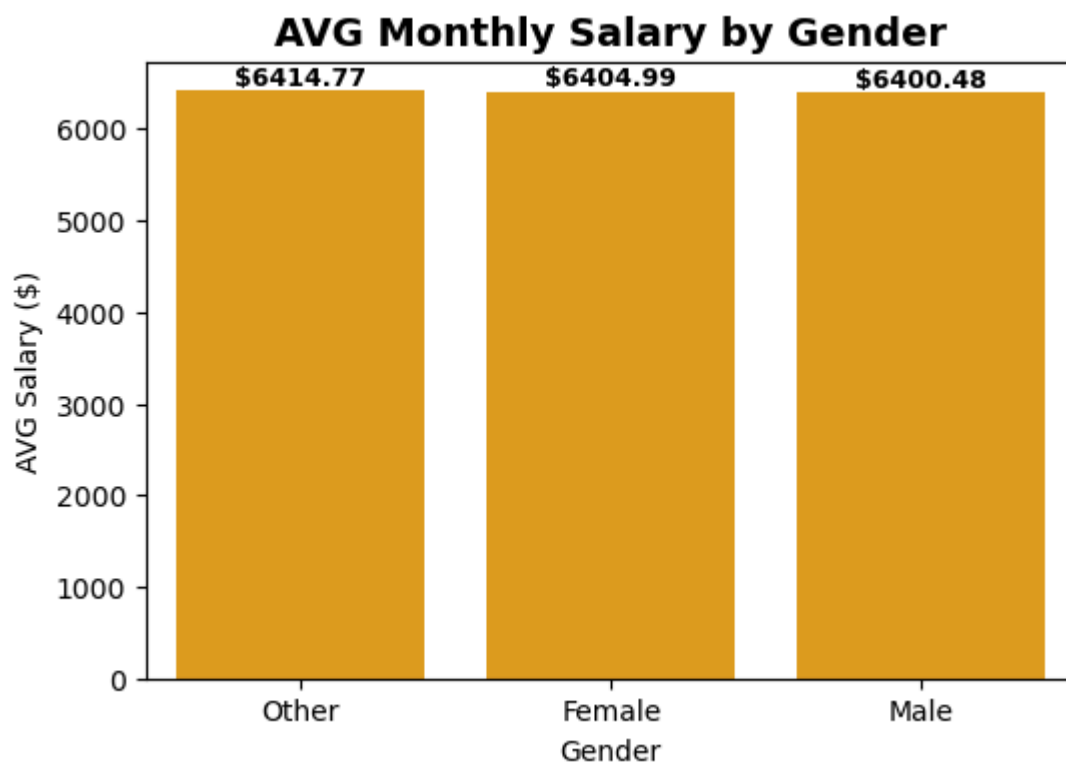
plt.title(
    "AVG Monthly Salary by Gender",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel(
    "Gender"
)

plt.ylabel(
    "AVG Salary ($)"
)

plt.savefig("11_avg_monthly_salary_by_gender.png", dpi=300, bbox_inches="tight")
plt.show()

```



In [27]: `avg_per_gender=data.groupby("Gender")["Performance Score"].mean().sort_values(ascending=False)`
`avg_per_gender`

Out[27]:

	Gender	Performance Score
0	Other	3.002520
1	Male	2.999854
2	Female	2.990417

```

In [28]: # Evaluating average performance scores grouped by gender for fairness analysis.
plt.figure(figsize=(6,4))

ax=sns.barplot(
    x="Gender",
    y="Performance Score",
    data=avg_per_gender,
    legend=False,
    color="darkcyan"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.2f",
        padding=0,
        fontsize=9,
        fontweight="bold"
    )

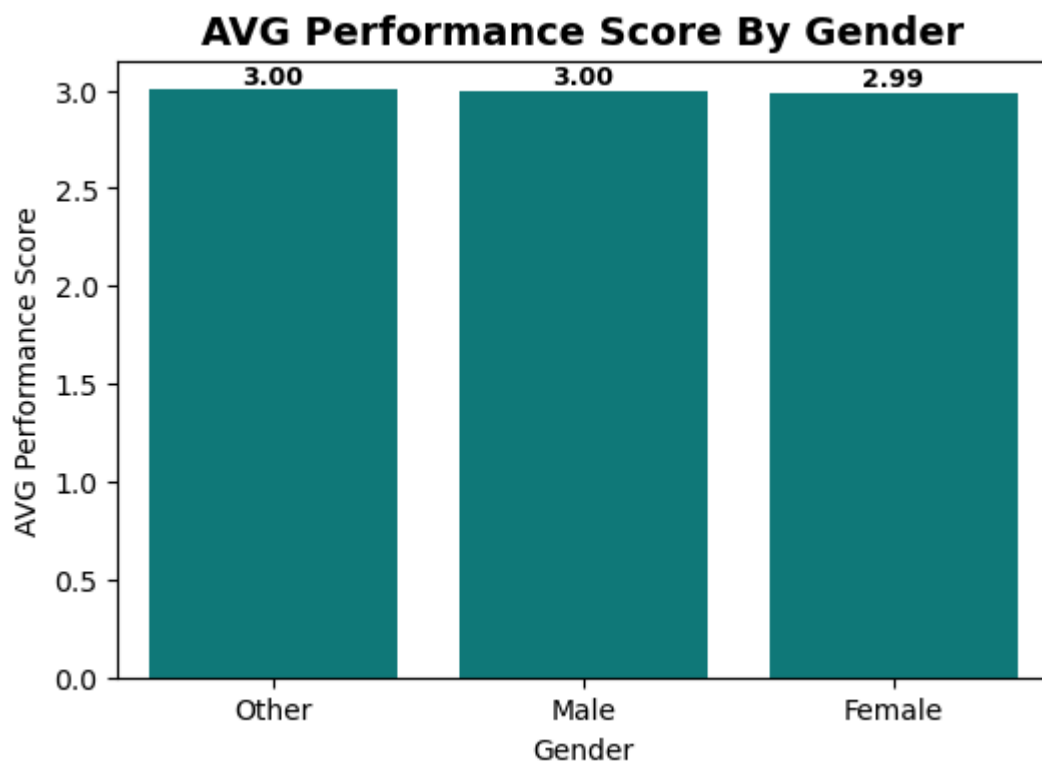
plt.title(
    "AVG Performance Score By Gender",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel(
    "Gender"
)

plt.ylabel(
    "AVG Performance Score"
)

plt.savefig("12_avg_performance_score_by_gender.png", dpi=300, bbox_inches="tight")
plt.show()

```



```

In [29]: # Plotting employee tenure distribution to understand workforce experience milestones.
plt.figure(figsize=(8,6))

ax=sns.histplot(

```



```

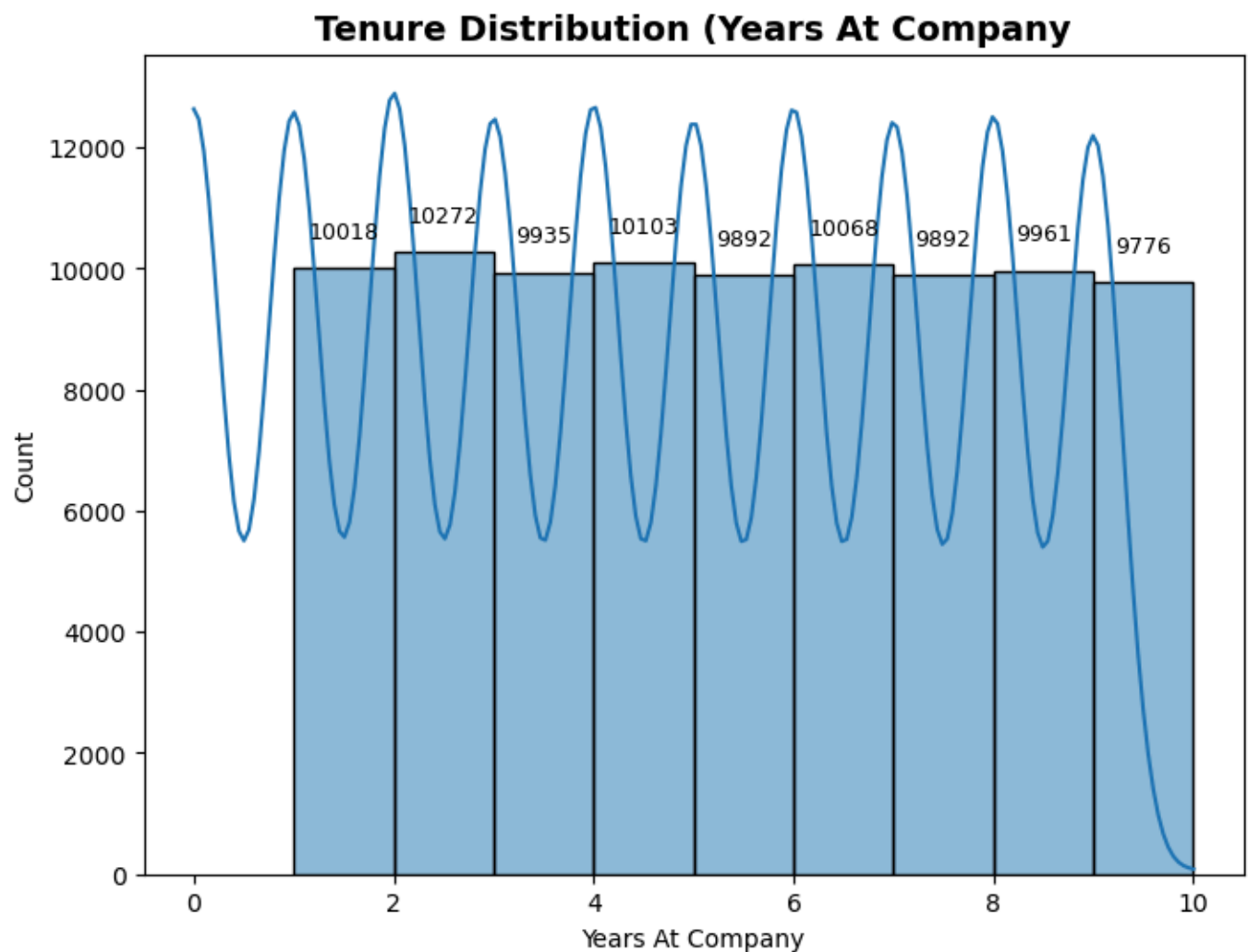
data=data,
x="Years At Company",
bins=[1,2,3,4,5,6,7,8,9,10],
kde=True
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=10,
        fontsize=9
    )

plt.title(
    "Tenure Distribution (Years At Company)",
    fontsize=14,
    fontweight="bold"
)

plt.savefig("13_tenure_distribution.png", dpi=300, bbox_inches="tight")
plt.show()

```



```
In [30]: experience_vs_salary=data.groupby("Years At Company")["Monthly Salary"].mean().reset_index(na
```

```
In [31]: experience_vs_salary
```

Out[31]:

	Years At Company	Salary
0	0	6401.566994
1	1	6390.147734
2	2	6420.380646
3	3	6406.713639
4	4	6403.385133
5	5	6406.798423
6	6	6394.338498
7	7	6412.338253
8	8	6397.274370
9	9	6399.064940
10	10	6347.727273

In [32]: *# Looking at how employee salary scales with years spent at the company.*

```
plt.figure(figsize=(12,6))

ax=sns.barplot(
    x="Years At Company",
    y="Salary",
    data=experience_vs_salary,
    legend=False,
    palette="flare"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="$%.0f",
        fontsize=9,
        fontweight="bold",
        padding=0
    )

plt.title(
    "Experience vs Salary",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel(
    "Years At Company"
)

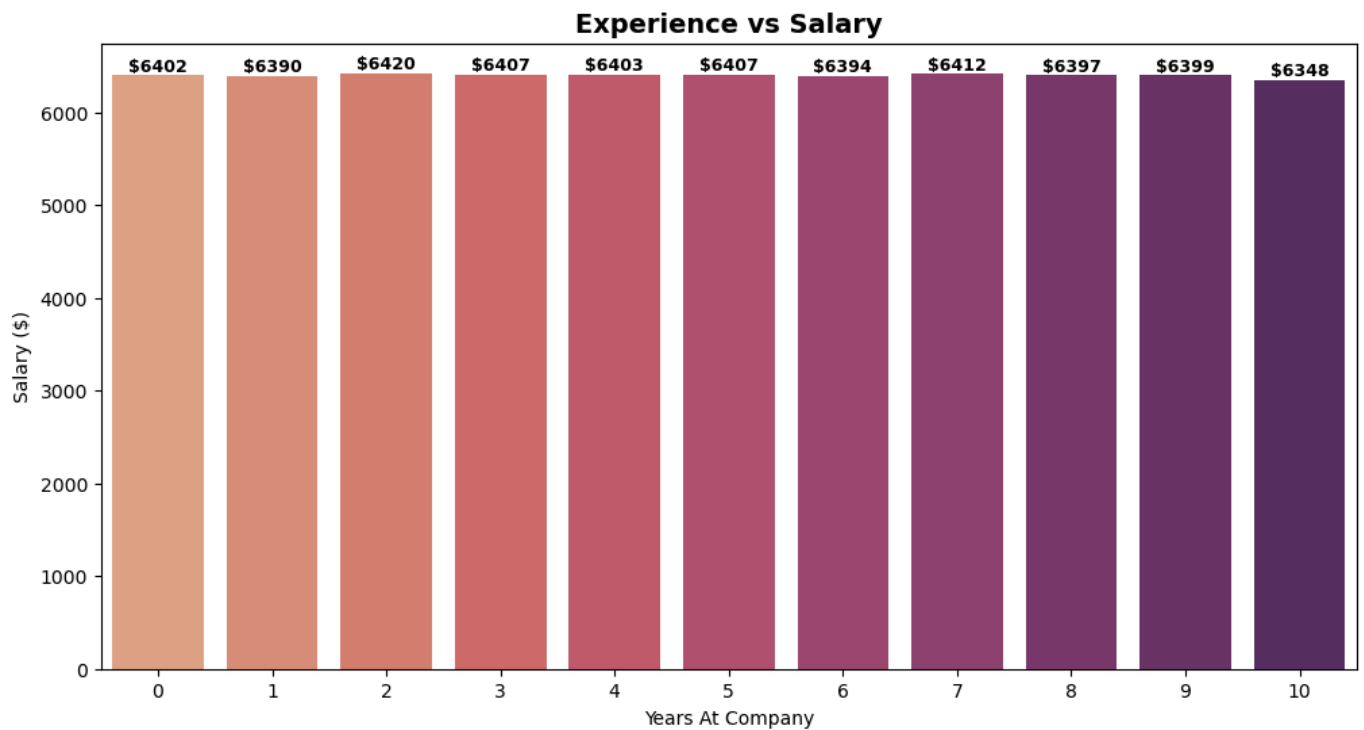
plt.ylabel(
    "Salary ($)"
)

plt.savefig("14_experience_vs_salary.png", dpi=300, bbox_inches="tight")
plt.show()
```

C:\Users\Dell\AppData\Local\Temp\ipykernel_7304\415171359.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax=sns.barplot(
```



```
In [33]: # Converting hire dates into standard datetime format and extracting the hiring year.  
data["Hire Date"]=pd.to_datetime(data["Hire Date"], format="mixed")  
data["Year"]=data["Hire Date"].dt.year  
data["Hire Date"]=data["Hire Date"].dt.strftime("%d-%m-%Y")
```

```
In [34]: hire_trend=data.groupby("Year")["Hire Date"].count().reset_index(name="Employees")
```

```
In [35]: hire_trend
```

```
Out[35]:
```

	Year	Employees
0	2014	3097
1	2015	9886
2	2016	9874
3	2017	10004
4	2018	10005
5	2019	9936
6	2020	10096
7	2021	10094
8	2022	10145
9	2023	9973
10	2024	6890

```
In [36]: # Plotting annual hiring trends over the years to track company growth velocity.  
plt.figure(figsize=(12,6))  
  
sns.lineplot(  
    x="Year",  
    y="Employees",  
    data=hire_trend,  
    legend=False,  
    marker="o",  
    linewidth=3,
```

```

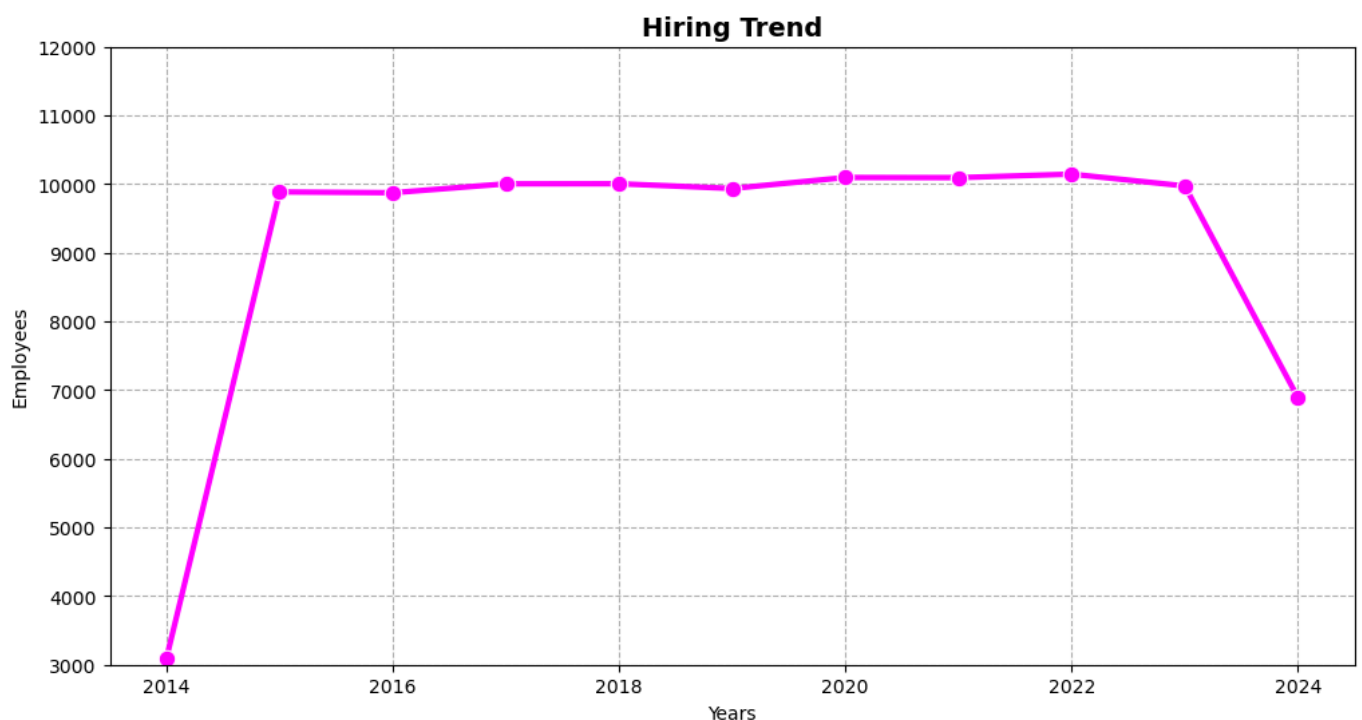
        markersize=9,
        color="magenta"
    )
plt.grid(True, linestyle="--", alpha=1)

plt.title(
    "Hiring Trend",
    fontsize=14,
    fontweight="bold"
)
plt.ylabel(
    "Employees"
)
plt.xlabel(
    "Years"
)

plt.ylim(3000,12000)

plt.savefig("15_hiring_trend.png", dpi=300, bbox_inches="tight")
plt.show()

```



In [37]: *#Average Performance by job title*
#Average Performance by education

In [38]: *# Analyzing attrition rates segmented by educational qualification levels.*

```

plt.figure(figsize=(8,5))
attrition_rate_edu=(data.groupby("Education Level")["Resigned"].sum()/
                    data.groupby("Education Level")["Resigned"].count()*100).sort_values(ascending=

ax=sns.barplot(
    x="Education Level",
    y="Attrition_Rate",
    data=attrition_rate_edu,
    legend=False,
    color="magenta"
)

for i in ax.containers:
    ax.bar_label(
        i,

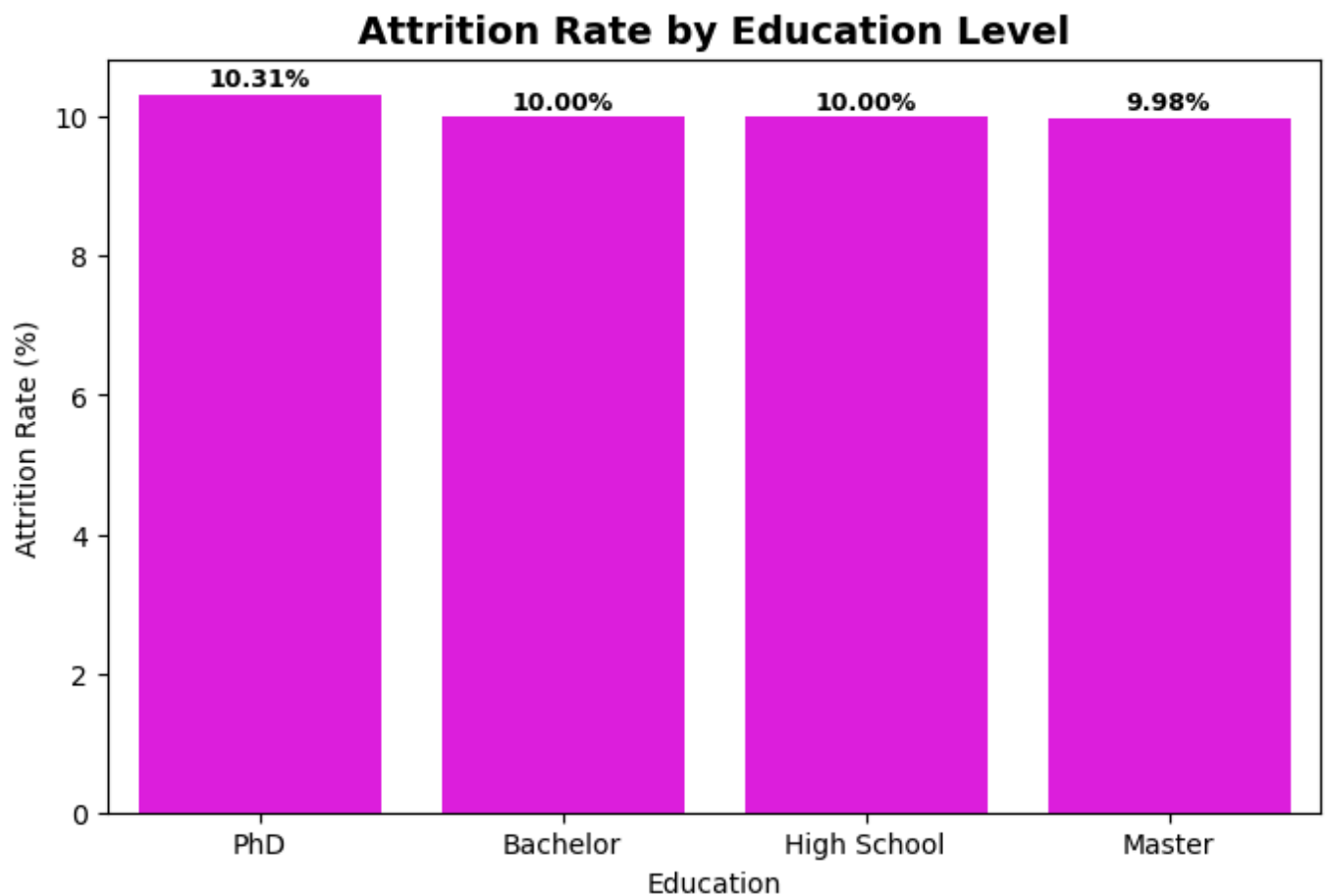
```

```

        fmt="%.2f%",
        padding=1,
        fontweight="bold",
        fontsize=9
    )

plt.title(
    "Attrition Rate by Education Level",
    fontweight="bold",
    fontsize=14
)
plt.xlabel(
    "Education"
)
plt.ylabel(
    "Attrition Rate (%)"
)
plt.savefig("16_attrition_rate_by_education.png", dpi=300, bbox_inches="tight")
plt.show()

```



```

In [39]: # Comparing average performance scores across different job designations.
plt.figure(figsize=(10,5))
performance_score_job=data.groupby("Job Title")["Performance Score"].mean().sort_values(ascending=True)

ax=sns.barplot(
    x="Job Title",
    y="performance",
    data=performance_score_job,
    legend=False,
    color="crimson"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.2f",
        padding=0,

```

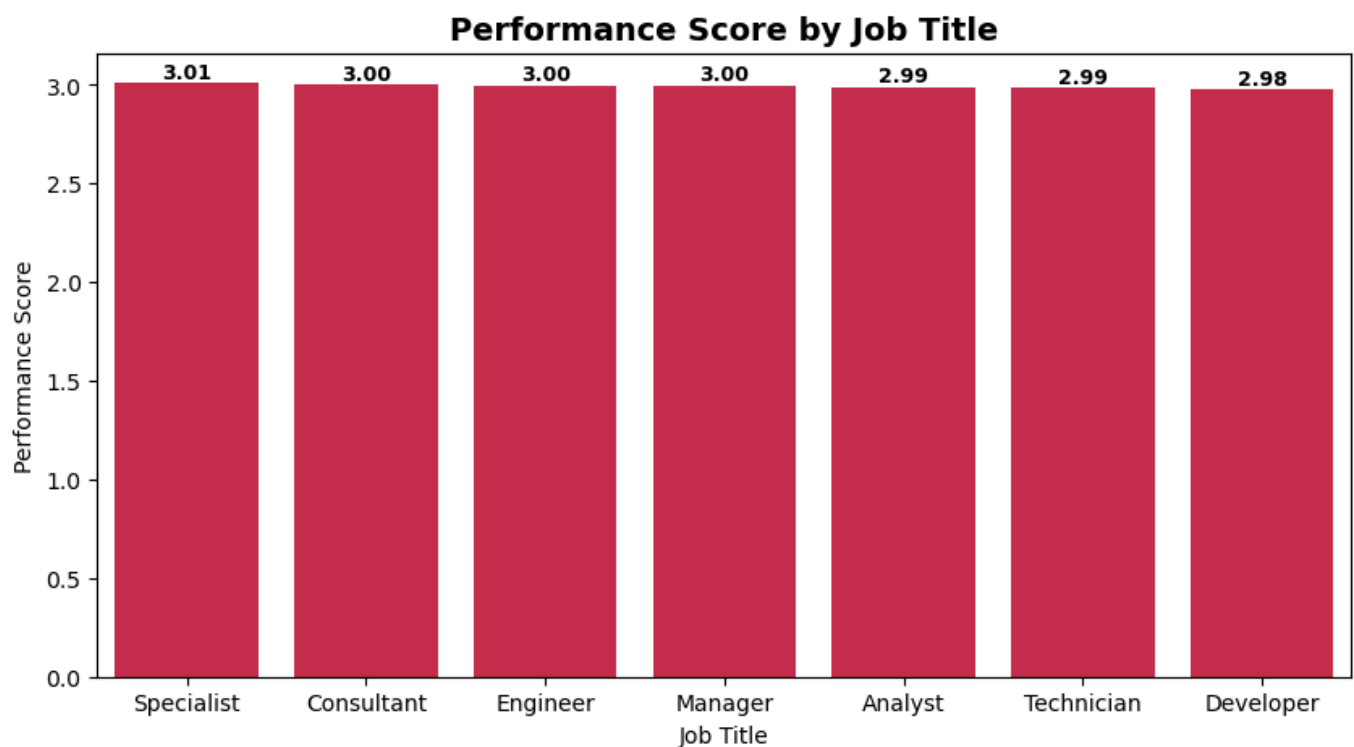
```

        fontweight="bold",
        fontsize=9
    )

plt.title(
    "Performance Score by Job Title",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Job Title"
)
plt.ylabel(
    "Performance Score"
)

plt.savefig("17_performance_score_by_jobtitle.png", dpi=300, bbox_inches="tight")
plt.show()

```



```

In [40]: # Checking performance score variations across different education tiers.
plt.figure(figsize=(8,5))
performance_score_education=data.groupby("Education Level")["Performance Score"].mean().sort_

ax=sns.barplot(
    x="Education Level",
    y="performance",
    data=performance_score_education,
    legend=False,
    color="cyan"
)

for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.2f",
        padding=0,
        fontweight="bold",
        fontsize=9
    )

plt.title(

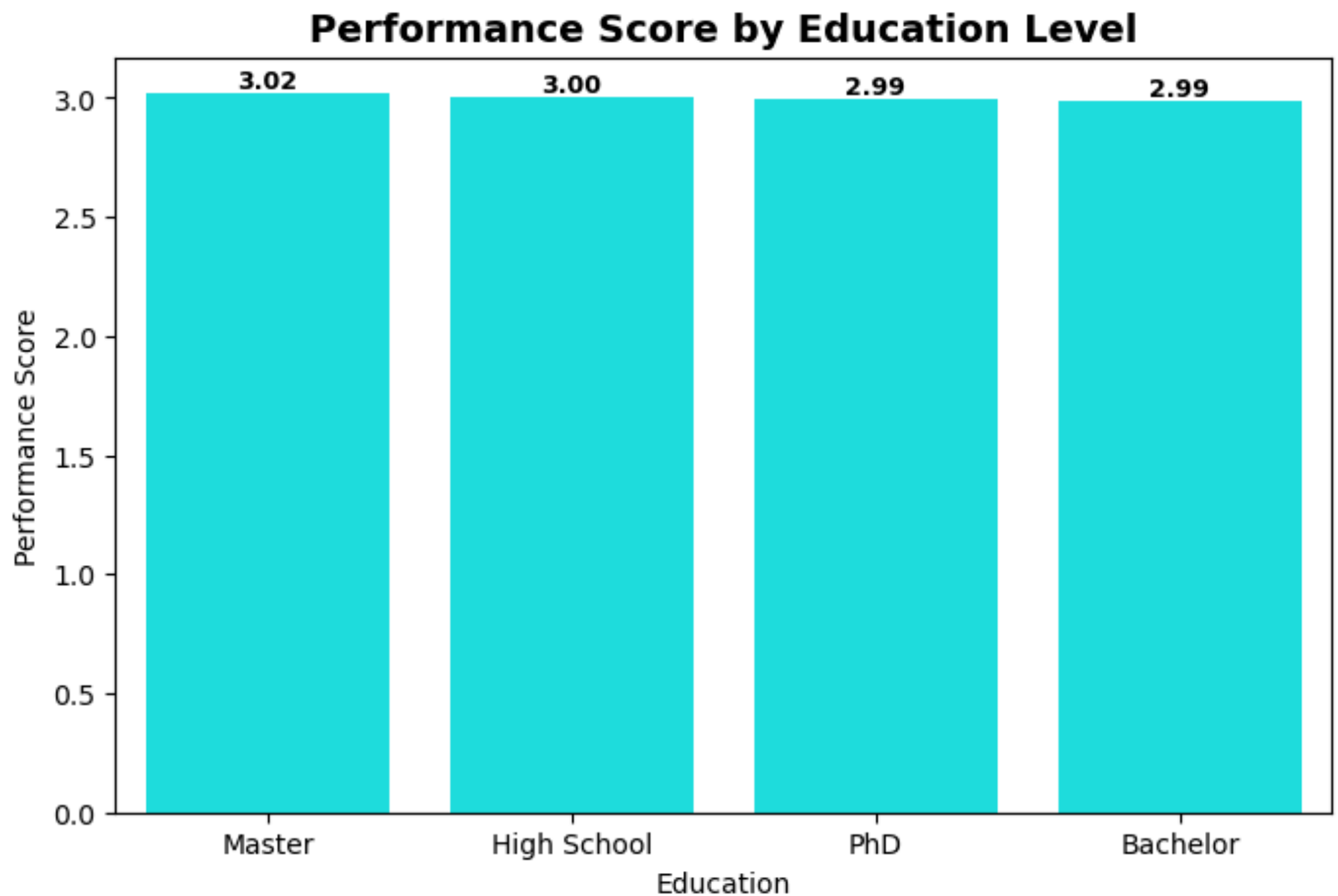
```

```

    "Performance Score by Education Level",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Education"
)
plt.ylabel(
    "Performance Score"
)

plt.savefig("18_performance_score_by_education_level.png", dpi=300, bbox_inches="tight")
plt.show()

```



```
In [41]: data.columns
```

```

Out[41]: Index(['Employee ID', 'Department', 'Gender', 'Age', 'Job Title', 'Hire Date',
               'Years At Company', 'Education Level', 'Performance Score',
               'Monthly Salary', 'Work Hours Per Week', 'Projects Handled',
               'Overtime Hours', 'Sick Days', 'Remote Work Frequency', 'Team Size',
               'Training Hours', 'Promotions', 'Employee Satisfaction Score',
               'Resigned', 'Satisfaction Range', 'Resignation', 'Year'],
              dtype='object')

```

```

In [42]: # Multi-plot overview analyzing relationships between overtime, work hours, salary, and satis;
fig, axs=plt.subplots(2,2,figsize=(16,12))
fig.suptitle(
    "Employee Performance & Satisfaction Analysis",
    fontsize=18,
    fontweight="bold",
    y=1
)

sns.scatterplot(
    x="Overtime Hours",
    y='Employee Satisfaction Score',

```

```

        data=data,
        alpha=0.1,
        color="blue",
        s=15,
        ax=axes[0,0]
    )
    axes[0,0].set_title(
        "Overtime vs Employee Satisfaction",
        fontsize=14,
        fontweight="bold"
    )
    axes[0,0].set_ylabel(
        "Employee Satisfacton Score"
    )
    axes[0,0].set_xlabel(
        "Overtime Hours"
    )

sns.scatterplot(
    x="Work Hours Per Week",
    y='Monthly Salary',
    data=data,
    alpha=0.1,
    color="blue",
    s=15,
    ax=axes[0,1]
)
axes[0,1].set_title(
    "Works Hour vs Monthly Salary",
    fontsize=14,
    fontweight="bold"
)
axes[0,1].set_ylabel(
    "Monthly Salary ($)"
)
axes[0,1].set_xlabel(
    "Work Hours"
)

sns.scatterplot(
    x="Performance Score",
    y='Monthly Salary',
    data=data,
    alpha=0.1,
    color="blue",
    s=15,
    ax=axes[1,0]
)
axes[1,0].set_title(
    "Monthly Salary vs Performance Score",
    fontsize=14,
    fontweight="bold"
)
axes[1,0].set_ylabel(
    "Monthly Salary ($)"
)
axes[1,0].set_xlabel(
    "Performance Score"
)

sns.scatterplot(

```



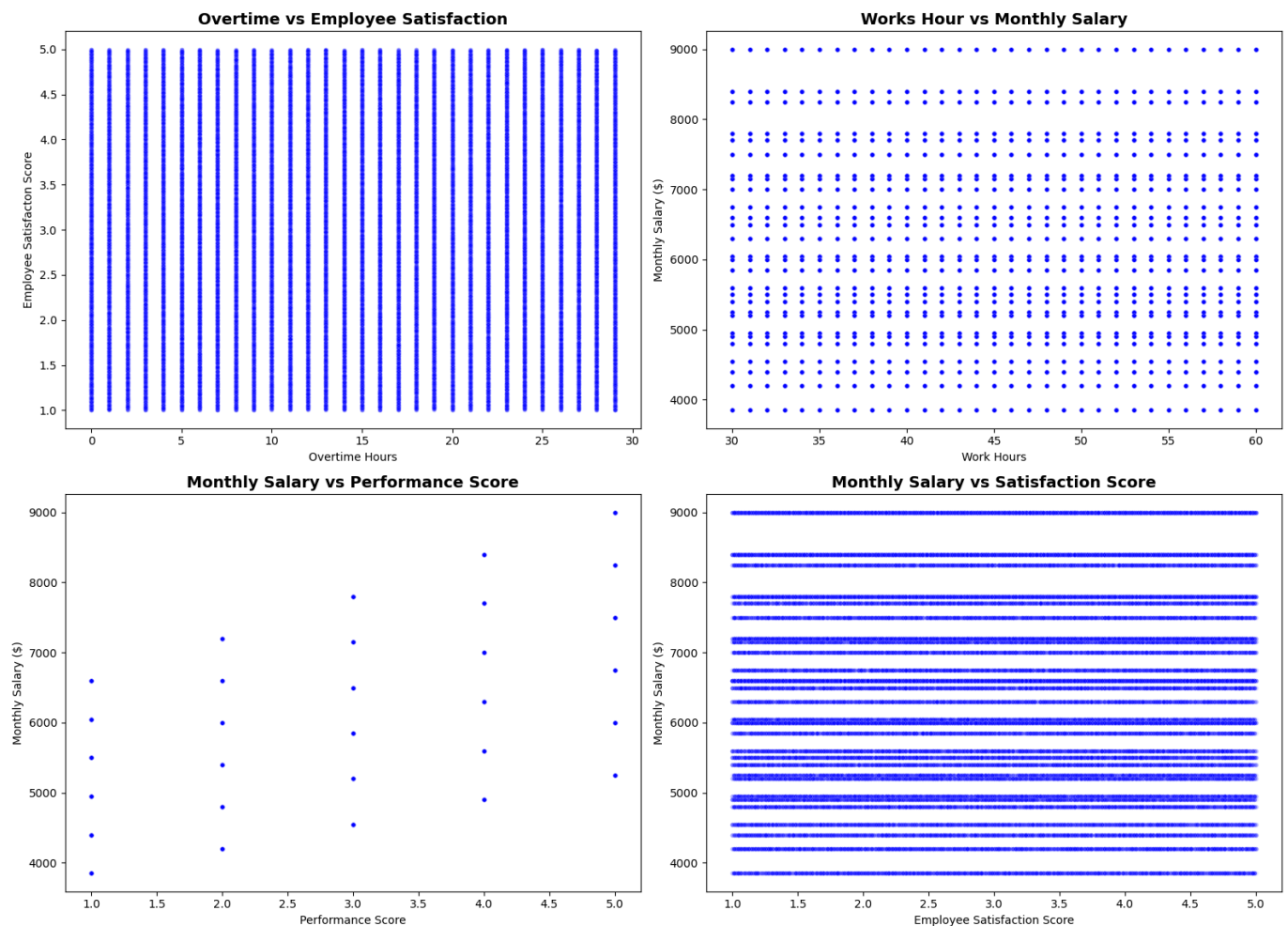
```

x="Employee Satisfaction Score",
y='Monthly Salary',
data=data,
alpha=0.1,
color="blue",
s=15,
ax=axes[1,1]
)
axes[1,1].set_title(
    "Monthly Salary vs Satisfaction Score",
    fontsize=14,
    fontweight="bold"
)
axes[1,1].set_ylabel(
    "Monthly Salary ($)"
)
axes[1,1].set_xlabel(
    "Employee Satisfaction Score"
)

plt.savefig("19_employee_performance_&_satisfaction_analysis.png", dpi=300, bbox_inches="tight")
plt.tight_layout()
plt.show()

```

Employee Performance & Satisfaction Analysis

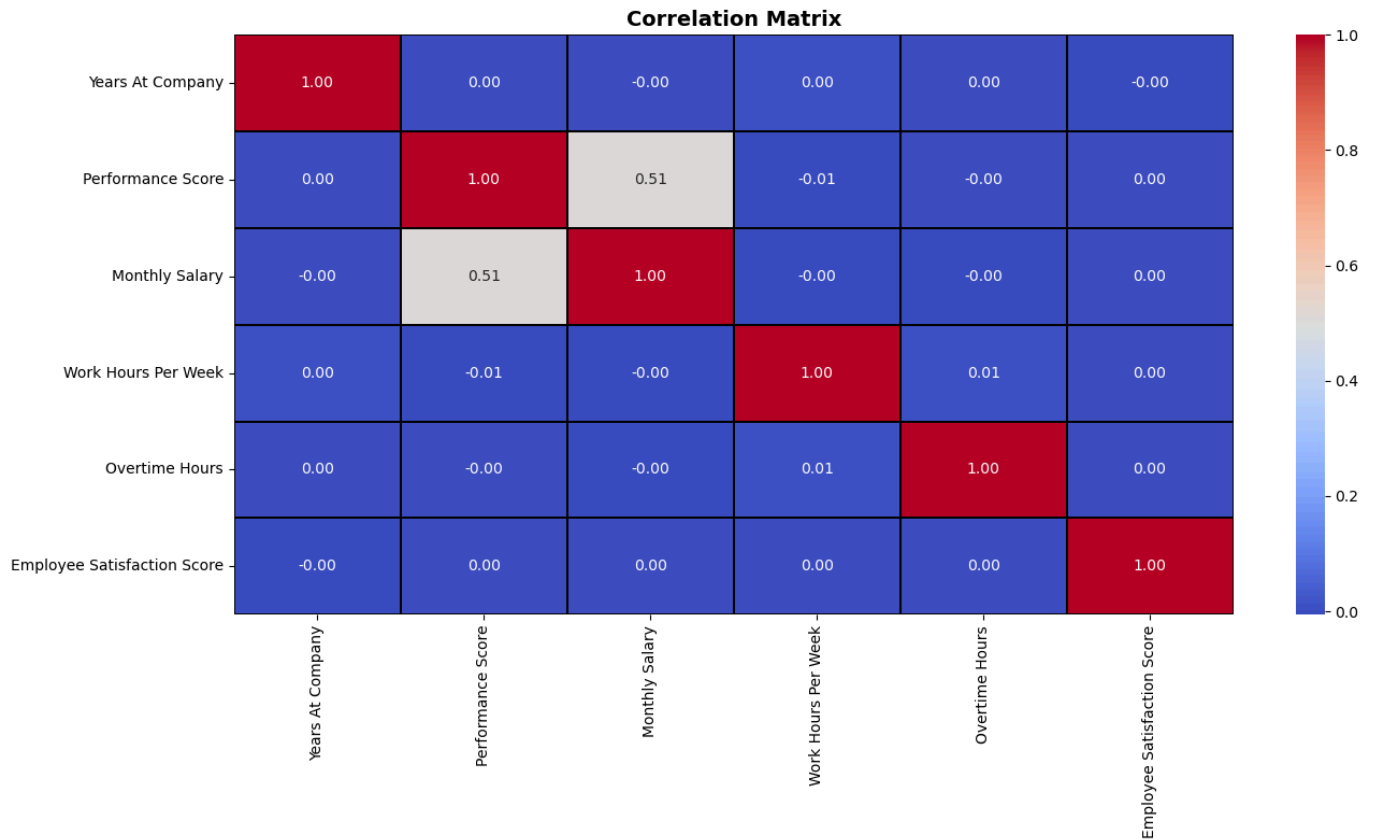


In [43]: *# Final review of dataframe structure and data types before moving to correlation analysis.*
data.dtypes

```
Out[43]: Employee ID          int64
          Department         object
          Gender             object
          Age               int64
          Job Title          object
          Hire Date          object
          Years At Company   int64
          Education Level    object
          Performance Score  int64
          Monthly Salary     float64
          Work Hours Per Week int64
          Projects Handled   int64
          Overtime Hours     int64
          Sick Days          int64
          Remote Work Frequency int64
          Team Size          int64
          Training Hours     int64
          Promotions         int64
          Employee Satisfaction Score float64
          Resigned          bool
          Satisfaction Range category
          Resignation        object
          Year              int32
          dtype: object
```

```
In [44]: # Comprehensive correlation heatmap covering key numerical features in the dataset.
plt.figure(figsize=(14,8))
num_cols=["Years At Company", "Performance Score", "Monthly Salary", "Work Hours Per Week", "Projects Handled", "Overtime Hours", "Sick Days", "Remote Work Frequency", "Team Size", "Training Hours", "Promotions", "Employee Satisfaction Score"]
corr=data[num_cols].corr()
sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt='.2f',
    linewidths=0.1,
    linecolor="black"
)
plt.title(
    "Correlation Matrix",
    fontsize=14,
    fontweight="bold"
)

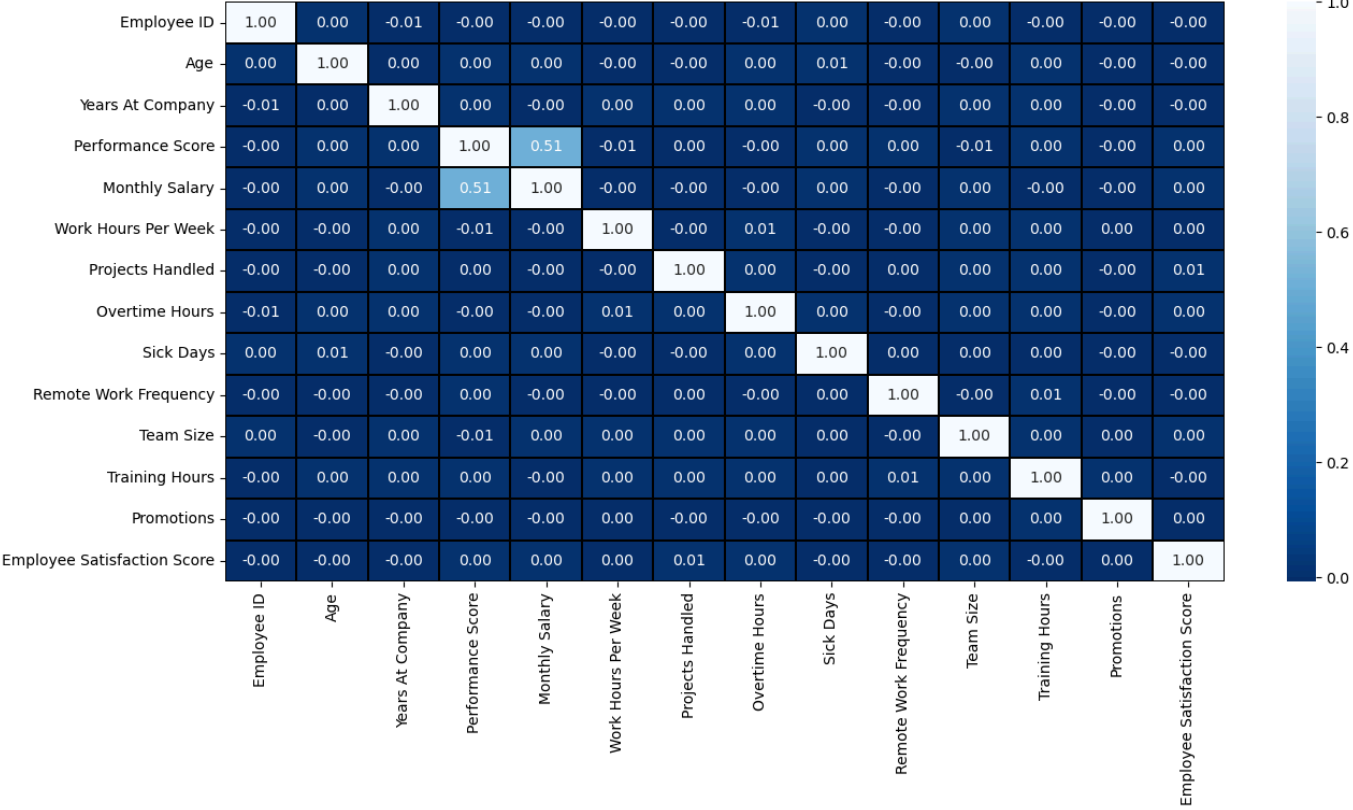
plt.savefig("20_correlation_matrix.png", dpi=300, bbox_inches="tight")
plt.tight_layout()
plt.show()
```



```
In [45]: # Comprehensive correlation heatmap covering key numerical features in the dataset.
plt.figure(figsize=(14,8))
num_cols=data.select_dtypes(include=["float64", "int64"]).columns
corr=data[num_cols].corr()
sns.heatmap(
    corr,
    annot=True,
    cmap="Blues_r",
    fmt='.2f',
    linewidths=0.1,
    linecolor="black"
)
plt.title(
    "Correlation Matrix of Numerical Features",
    fontsize=14,
    fontweight="bold"
)

plt.savefig("21_correlation_matrix_of_numerical_feature.png", dpi=300, bbox_inches="tight")
plt.tight_layout()
plt.show()
```

Correlation Matrix of Numerical Features



In []: